Delay Insensitive Logic for RSFQ Superconductor Technology

Priyadarsan Patra
Intel Corporation
5200 NE Elam Young Pkwy.
Hillsboro, OR 97124

Stanislav Polonsky*
Physics Department
SUNY Stony Brook
Stony Brook, NY 11794

Donald S. Fussell
Department of Computer Sciences
The University of Texas at Austin
Austin, TX 78712

Abstract

Asychronous designs have been touted as having potential advantages in average performance, power consumption, modularity, and tolerance of metastability as compared to traditional synchronous logic. While delay-insensitive (DI) asynchronous circuits are theoretically the most desirable type of asynchronous logic because they make the weakest timing assumptions, the complexity of implementing DI circuits in CMOS or similar technologies may make them impractical to use.

The fact that event-based DI circuits are ill matched to CMOS does not necessarily mean that they are inherently inefficient, however. In this paper, we show that using Rapid Single Flux Quantum (RSFQ) superconducting circuits, in which information is represented as discrete voltage pulses or magnetic flux quanta, many powerful DI circuit primitives can be implemented at least as efficiently as Boolean logic gates. Since DI logic also alleviates the severe clock skew problems that can be expected at the switching speeds approaching a terahertz in this technology, it may well be a more practical basis for digital circuit design than alternatives traditionally used for CMOS.

1. Introduction

Asynchronous circuits have received increasing interest in academia and industry in recent years as power dissipation becomes an increasingly limiting factor with rise in circuit density and speed in CMOS. Asynchronous circuits promise to be inherently faster than synchronous circuits since they operate at the average speed of the operations in the circuit rather than the worst case speed. They promise to use less power because only those portions of the circuit that are active in a computation switch at any given point in time. They also promise to be easier to compose modularly than synchronous circuits and easier to implement at high speeds when clock skew might otherwise become a problem. This is particulary true for delay-insensitive (DI) circuits, since they are guaranteed to operate correctly in the face of arbitrary finite delays in logic elements and the interconnect.

DI circuits, on the other hand, may not be able to realize any practical gains in power dissipation or performance simply due to their complexity of implementation in CMOS. A primitive DI circuit element can take at least 40–50 transistors to implement, as compared with the 4 required for a two-input Boolean logic gate used in synchronous circuits. This extra logic is needed because DI circuits employ a logic of events, requiring dual rail signaling and handshaking logic and requiring many primitives to be stateholding elements. CMOS and related conventional technologies are based on a logic of values represented as voltage levels, making them ill-suited to the implmentation of DI circuits.

Some promising new technologies for the implementation of high speed circuits are event-based, however. One such technology is Rapid Single Flux Quantum (RSFQ) logic [7], which is based on low temperature superconductors using Josephson junctions (JJ) as the basic switching elements. In RSFQ logic circuits, very short pulses, which can equivalently be thought of as single magnetic flux quanta, carry the information. Existing primitives in RSFQ logic are naturally event-based, pulses are generated within a primitive and are released to the external world on the arrival of a clock. Josephson-junction based RSFQ circuits can operate at frequencies of hundreds of gigahertz, with the potential to extend into the terahertz range, while at the same time consuming several orders of magnitude less power than semiconductor circuits and allowing signal propagation at the speed of light. They are actually simpler to fabricate than semiconductor technologies at similar feature sizes, but they have the drawback that they require liquid helium-based cryocooling. Superconducting circuits are in use today for specialized applications, and fabrication of 10,000 Josephson junction integrated systems with reasonable yields is common.

*This author's work was supported by the DoD's University Research Initiative (AFOSR grant #F49620-95-I-0415).

0-8186-7922-0/97 $10.00 © 1997 IEEE

Recently, there have been some initial attempts to design DI circuits in RSFQ. In [11] and [12], the suitability of asynchronous computation in ultra speed Josephson junction technology was proposed along with the idea of superposition of persistent currents to build logic. The use of a dual-rail input validity detection signal as the "clock" signal to turn the "clocked" RSFQ circuits of [7] into asynchronous circuits was reported in [6], [8], and [10]; however, the resulting DI designs were expensive and involved some non local timing assumptions. A similar "self-timed" logic family based on clock signal generation upon arrival of valid data was described in [1].

In this paper, we argue that DI circuits are actually more naturally suited to implementation in RSFQ than are conventional, Boolean logic based synchronous circuits [11]. We show efficient RSFQ implementations of some of the primitive elements from a *universal* set of DI primitives, that is a set from which arbitrary state machines can be implemented using only these primitive circuit elements. These implementations are shown to be at least as efficient in terms of the number of Josephson junctions used, the layout space required, the maximum switching delays required, and the noise margins of the circuits, as standard RSFQ implementations of Boolean logic gates from [7]. Interestingly, a few of these primitives already correspond to RSFQ circuits described in [7].

The remainder of the paper is organized as follows. First, we present a background on DI circuits, along with our set of universal primitive elements. Next, we present a brief introduction to RSFQ technology, followed by our main results, in which we show designs, simulation and implementations of our primitives as RSFQ circuits, including test results from some fabricated circuits. We then give a simple design example of a DI serial adder to show how such a device would be implemented with our proposed logic family. Finally, we discuss the next steps involved in scaling these results up to larger RSFQ systems.

2. Asynchronous and DI Systems

We are concerned with *delay-insensitive* asynchronous circuits. A *delay-insensitive (DI)* system is one whose specified functional behavior is independent of any finite delays in the subsystems or in the wires interconnecting the subsystems' terminals. A DI module or system is a hardware process with a well-defined set of allowable sequences of input and output events at its I/O ports.

We use *Trace Theory* (see, e.g., [16]) to specify the behavior of DI modules. A *trace structure* TS is a triple $< I, O, T >$ where I and O denote disjoint input and output alphabets, respectively, of the system, and $T \subseteq (I \cup O)^*$ is the *trace set* of all allowable *traces*, or sequences of symbols from $\{I \cup O\}$. In a mechanistic interpretation of TS, the symbols in input/output alphabets correspond to the circuit's input/output ports. Each symbol in a trace represents the occurrence of an event (sometimes called an action) at the port corresponding to the symbol. Normally we assume implicit, disjoint sets of input and output symbols: we append '?' or '!' to a symbol name to denote an input or output action, respectively. We may leave these suffixes out for internal signals or when no confusion is to be expected.

Symbol names in a trace structure can also be viewed as atomic *trace-structures* representing simple specifications. A more complex specification is built recursively from the primitive specifications by applying following operations.[1] The 1-place **pref** operation, when applied to a set of behaviors, represents all prefixes of those behaviors. All module behaviors are prefix-closed; all partial interface behaviors (i.e., communications) that lead to an admissible behavior are themselves admissible. The sequential composition of u and v, i.e., uv, denotes that behavior v necessarily follows behavior u. The 1-place '$\star$' operation denotes all finite concatenations (i.e., sequential compositions) of the behaviors in its argument set. A symbol s raised to a (natural) N indicates sequential composition of N such symbols. The 2-place parallel composition operation '$||$' denotes the set of *all* behaviors satisfying the following:

1. It has only those symbols that appear in the implicit input or output sets of either argument.
2. When it is *restricted* (or projected) to the set of implicit input *and* output symbols of either argument behavior set, the result must be a legal behavior in that argument set.

This operation models the concurrency of causally unrelated events in its two argument sets. There is no notion of simultaneity in DI models due to the possibility of arbitrary communication delays. The 2-place '$|$' operation denotes the union of the two argument sets of behaviors: e.g., $S = a(b \mid c)$, where all the symbols are either inputs or outputs, specifies that after a, either b or c, but not both, is allowed; thus the set of valid traces is $\{a\, b, a\, c\}$. [2]

3. Universal DI Primitives

The Muller C-element is perhaps the most widely recognized DI primitive, and various researchers have considered extending the Boolean logic basis of conventional computer design to DI designs by adding the C-element

[1] For a good introduction to trace theory for specifying circuits, see [2].

[2] This trace-structure is not *delay-insensitive* [16], because it does not contain the trace ba, although ab is a valid trace, and both the symbols are inputs or outputs (hence, not *causally* related). The prefix-closure of S is $\{\epsilon,\ a,\ a\, b,\ a\, c\}$, so S also fails to meet the prefix closure requirement of DI trace structures.

primitive. However, it is well known that the class of DI circuits that can be implemented using only C-elements and Boolean logic gates is quite small [9]. This limitation need not exist for circuits constructed from a more robust set of DI primitives, however. In 1974, Keller [5] characterized a class of delay-insensitive(DI) circuits which is essentially equivalent to the class of finite state machines realizable as synchronous circuits, and he also provided a **universal** set of circuit primitives such that any circuit in this class is realizable as a delay-insensitive network of the primitives.

Our work is based on a new set of primitive modules [11] for delay-insensitive circuit design which are in a number of ways more efficient than Keller's primitives. These are shown in Table 1. Each primitive's specification is shown next to it in the form of a trace-theoretic specification as described above. We have shown [11] that the set 2×1-*Join*, *Mem* } and {*Fork*, *Merge*, *Mutex*, *Tria* } is *universal* for Keller's class of DI modules and that it is *minimal* in that no proper subet of it universal.

An $m\times n$-*Join* is operationally described as follows: It has m row inputs, n column inputs, and a matrix of $m \times n$ outputs—one for each pair of row and column inputs. The device and its environment repeat the following behavior. The device waits to receive exactly one row input and exactly one column input; upon receiving the two inputs it makes a transition on the output corresponding to the input pair. (*Joins* are equivalent under swapping of the row and the column inputs.) A 1×1-*Join* is identical to a C-element.

As shown in Table 1, the trace-theoretic specification of a 1×2-*Join* (equivalently 2×1-*Join*) is given by: $\mathbf{pref}(((a?||b0?)\ c0!)\ \mid\ ((a?||b1?)c1!))^{\star}$. The symbols a, $b0$, and $b1$ with the suffix '?' represent input events, and the symbols $c0$ and $c1$ followed by '!' represent output actions. Hence, the input set is $\langle\, a, b0, b1\rangle$ and the output set is $\langle\, c0,\ c1\rangle$. Examples of valid partial behaviors are: a, $a\,b0$, $a\,b0\,c0$, $a\,b0\,c0\,b0$, $b0\,a\,c0$, $b1\,a\,c1\,a\,b1\,c1$, $a\,b1\,c1\,b0\,a\,c0$. Some invalid traces are: (1) $a\,b0\,b1$, (2) $a\,a$, (3) $b0\,c0$, (4) $a\,b0\,c1$. The first two traces represent errors in the environment, while the last two are module errors (refer to the operational description of a *Join*). In (1) the environment cannot send both column inputs $b1$ and $b0$ without an intermediate output from the *Join* and in (2) a cannot immediately follow itself for the same reason. In (3) output $c0$ is produced too early, while in (4) $c1$ is the wrong output, $c0$ should be produced instead.

A *Tria* outputs an event on a vertex when it has received events on the two edges adjacent to that vertex. A *Fork*, represented by branched lines, repeatedly copies each input event to both of its output ports. A *Merge*, usually implemented as an Xor gate in CMOS, repeats the following: it can receive exactly one event on either of the input ports and, upon reception, copies it to the output port. A *Toggle* device repeatedly copies an input event to an output before getting ready to receive the next input, but the output events are distributed between the two output ports alternately. Note that the *Toggle* can be implemented as a special case of 1×2-*Join*. The *Mutex* element provides mutual exclusion using 4-phase handshaking [11], while the *Sequencer* provides two-phase arbitration [11] with the help of an extra input signal. Note that the *Sequencer* can be implemented using the other primitives, but it is significantly more complex than *Mutex* when implemented this way.

A 'bubble' at an input terminal of a *Join* device implies an initialization that corresponds to a state where a transition at that terminal is assumed to have been received initially. We can alternatively imagine a *Merge* gate at the bubbled input that has an input port connected to a 'START' signal. For instance, the behavior of a C-element with a bubble at the a-input is a device with the behavior of a 1×1-*Join* after receiving a transition on a first, i.e., $\mathbf{pref}(b?\ c!((a?||b?)c!))^{\star}$. A heavy dot near a terminal of a device denotes an initialization where only the thus indicated terminal may produce the first output of the device. We occasionally use a circle labeled with 'P' as a short-hand for a tree of *Merges* in our figures.

4. Overview of RSFQ Logic

RSFQ logic [7] is based on Josephson junction (JJ) devices in low temperature superconductors. In current technology, a JJ consists of a pair of niobium superconductor electrodes separated by aluminum oxide as a thin tunnel barrier. Below a certain critical current I_c, such junctions carry *superconducting current* with no need for a bias voltage across the junction, thus not dissipating any energy. When the induced current exceeds I_c, the junction becomes resistive and undergoes a *Josephson* 2π *phase leap* which produces a non-zero voltage drop across the junction. This effect in two-terminal Josephson junctions is the basis for the switching action needed to build logic devices.

RSFQ logic circuits use overdamped Josephson junctions instead of the underdamped junctions used in earlier projects. This allows the junctions to switch between superconductive and resistive states at speeds on the order of several hundred gigahertz and avoids the cumbersome global reset signal which limits IBM style "latching logic" circuits to clock frequencies of only a few gigahertz [7].

Superconductive latching logic circuits, like semiconductor transistor logics, represent information as dc voltage values. In RSFQ logic circuits, on the other hand, a bit of information is carried by the propagation of a single magnetic flux quantum $\Phi_0 \equiv \frac{\hbar}{2e} \simeq 2.07\times 10^{-15} Wb$, where $\hbar$ is Planck's constant and e is the charge of an electron, hence the name of the technology. These quanta can equivalently be thought of as very short voltage pulses $V(t)$ of quantized area, where $\Phi_0 = \int V(t)\ dt \simeq 2.07mV \times ps$. Pulse propa-

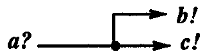
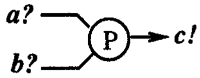
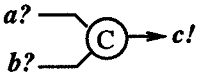
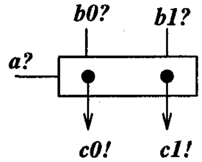
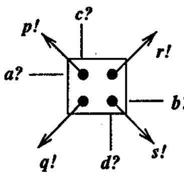
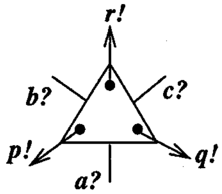
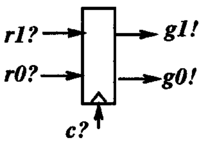
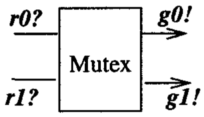
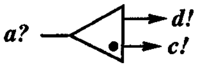
Name	Symbol	Specification	Name	Symbol	Specification
Fork		**pref** $(a?(\,b!\|\|\,c!))^*$	*Merge*		**pref** $((a?\ \|b?)\ c!)^*$
1×1-*Join*		**pref** $((a?\|\|b?)\ c!)^*$	1×2-*Join*		**pref** $(((a?\|\|b0?)\ c0!)$ $\|((a?\|\|b1?)c1!))^*$
2×2-*Join*		**pref** $(((a?\|\|c?)\ p!)$ $\|((b?\|\|c?)q!)$ $\|((a?\|\|d?)r!)$ $\|((b?\|\|d?)s!))^*$	*Tria*		**pref** $(((a?\|\|b?)\ p!)$ $\|((a?\|\|c?)q!)$ $\|((b?\|\|c?)r!))^*$
Sequencer		**pref** $((r0?g0!)^*$ $\|\|(r1?g1!)^*$ $\|\|(c?(g0!\|g1!))^*)$	*Mutex*		**pref** $((r0?g0!r0?g0!)^*$ $\|\|(r1?g1!r1?g1!)^*$ $\|\|((g0!g0!)\|(g1!g1!))^*)$
Toggle		**pref** $(a?\ c!\ a?\ d!)^*$			

Table 1. A set of representative DI Primitives

gation takes place by biasing the JJs in the circuit so that an arriving flux quantum will cause a JJ to exceed its critical current I_c, thereby switching and emitting a new flux quantum. A full flux quantum is generated whenever a JJ's I_c is exceeded, regardless of any degradation that may have occurred to the triggering quantum, thus providing for power gain in RSFQ circuits.

The key advantages of superconducting RSFQ technology include

- sub-picosecond junction switching speed,
- ballistic propagation of signals via readily impedance-matched superconducting microstrip lines, and
- very low power dissipation, typically below one microwatt per Josephson junction even in its resistive state.

Josephson junction fabrication technologies are considerably simpler than either silicon or gallium arsenide transistors with similar design rules, and physical limits of the junction size are close to those of semiconductor transistors. These advantages make superconducting circuits very attractive alternatives to semiconductor technologies for ultra-high-speed applications. The major apparent disadvantage is the use of low T_c superconductors which must be cryocooled using liquid helium. At present, such cooling technology can cost on the order of $10,000, which is certainly not prohibitive for use in specialized applications. Projected improvements in cooling technology lead to estimates of cryocoolers costing on the order of $1,000 being available in the near term, with even cheaper cooling being available with migration to liquid nitrogen cooled high T_c superconductors. Other limitations of RSFQ circuits include limitations on RSFQ memory density due to the large physical size of a flux quantum and the difficulty of amplifying output signals to off-chip power levels at speeds comparable to those attainable on the chip.

5. RSFQ Implementations of DI Primitives

In this section we will describe the principles of operation of RSFQ implementations of several of the DI primitives described above. We will begin with the simplest primitives, which were introduced (with different names and as part of a different logic family) in [7]. We then go on to describe our new RSFQ implementations of more sophisticated primitive elements in more detail, including simulation results and low frequency experimental results from fabricated circuits.

5.1. Previously Existing Implementations

Some indication of the natural match of our universal set of DI primitives to RSFQ technology is given by the fact that the "pulse splitter," "confluence buffer," and "coincidence junction" logic circuits given in [7] are implementations of the *Fork* and *Merge* and 1×1-*Join* primitives respectively. These relatively simple primitives provide basic examples of the principles of operation of RSFQ circuitry.

5.1.1. RSFQ Fork Element

In the fork or pulse splitter, junctions $J1$, $J2$ and $J3$ (marked as $\times$'s in the circuit diagram of Fig. 1) are each in their own superconducting loops with bias currents $Ib1$, $Ib2$, and $Ib3$ (indicated as current sources by the double circles), respectively. These bias currents are just below the critical current I_c that would cause a junction to go resistive. When an input pulse enters the circuit at the input a, it induces an additional current $Ia = \frac{\Phi_0}{L1}$ across inductor $L1$ such that the total current $Ia + Ib1$ through junction $J1$ exceeds I_c and $J1$ becomes resistive. The resulting voltage drop across $J1$ triggers an SFQ pulse which propagates to $J2$ and $J3$, tripping both of them and causing pulses to be output at b and c respectively.

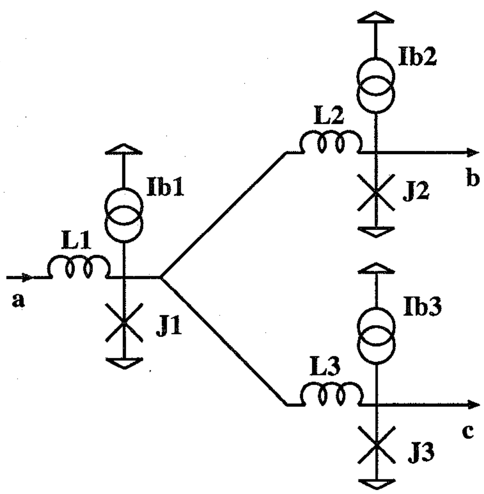


Figure 1. RSFQ implementation of *Fork*

5.1.2. RSFQ Merge Element

In the RSFQ implementation of *Merge* shown in Fig. 2, the two inputs are a and b, and the output is c. $Ib1$ is the bias current, which is split between two arms feeding junctions $J3, J1$ and $J4, J2$. The critical current thresholds are arranged so that $I_{c3} < I_{c1}$ and likewise $I_{c4} < I_{c2}$. When a pulse on a arrives, additional current flows through $J1$, exceeding its critical current, whereupon $J1$ goes resistive. $J3$ is not triggered since the current induced by the input pulse is in the opposite direction from the bias current. The SFQ pulse developed consequently across $J1$ is transferred through $J3$, across which the potential is still zero, to $J5$. This causes $J5$ to trip and emit an output pulse at c. The pulse generated by $J1$ also trips $J4$, whose critical current is less than that of $J2$. When $J4$ becomes resistive, it prevents $J2$ from tripping. Since there is no voltage drop across $J2$, no pulse is emitted back through input b. Inputs on b operate symmetrically. Thus the junctions $J3$ and $J4$ serve to isolate the inputs from each other and provide signal directionality (from inputs to output).

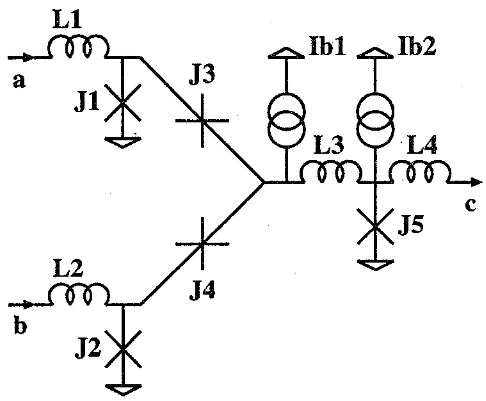


Figure 2. RSFQ implementation of *Merge*

5.1.3. RSFQ 1×1-*Join*

Also in [7], a circuit called a "coincidence junction" was presented, which implements a 1×1-*Join* or a C-element. The circuit for this device is depicted in Fig. 3. In this circuits, both junctions $J1$ and $J2$ are biased so that a pulse arriving on a or b respectively triggers one of them. What is different about this circuit compared to the previous ones is that it is a stateholding device. In RSFQ, bits are stored in "quantum interferometers" as superconducting current loops. There are two such interferometers in this circuit, the first is a loop including $J1, L1$ and $J3$ through ground and the second is a similar loop including $J2, L2$ and $J3$. If the inductance L of the interferometer is $L = \frac{2\pi\Phi_0}{I_c}$ and the bias current on one of the JJs is $0.8I_c$, then the interferometer has two symmetric stable stationary states corresponding to superconducting currents flowing or not in the loop and also corresponding to the presence or absence of a flux quantum in the loop. Inductances in a loop of the proper value to give a total loop inductance of L are called "quantizing inductances." $L1$ and $L2$ are quantizing inductances in this circuit for the two interferometers described above. Thus each of these interferometers can store a bit as a flux quantum. If $L1$ and $L2$ are sized to provide total loop inductances of L in the two interferometers above, the loop $(J1, L1, L2, J2)$ will not support stable superconducting currents and thus storage of a bit in this loop will be impossible because the total inductance in the loop will be wrong.

In the circuit's initial state, no persistent superconducting current is present in the interferometer loops. When an input arrives at a, $J1$ is triggered, but the pulse generated is insufficient to trigger $J3$. $J3$ remains superconducting, as is $J1$ once the pulse has been emitted. However, persistent superconducting current flowing through this loop from ground to $J1$ through $L1$ through $J3$ to ground has now been initiated, storing the flux quantum that arrived at a. This continues indefinitely through the superconducting medium in the absence of external interference. When a pulse arrives at b, $J2$ is tripped, and a second current loop through $J3$ is formed. The addition of the current from this second loop causes the total current through $J3$ to exceed the threshold of criticality, and $J3$ trips. This causes a pulse to be emitted at c and resets superconducting current in the loops, returning the circuit to its initial state.

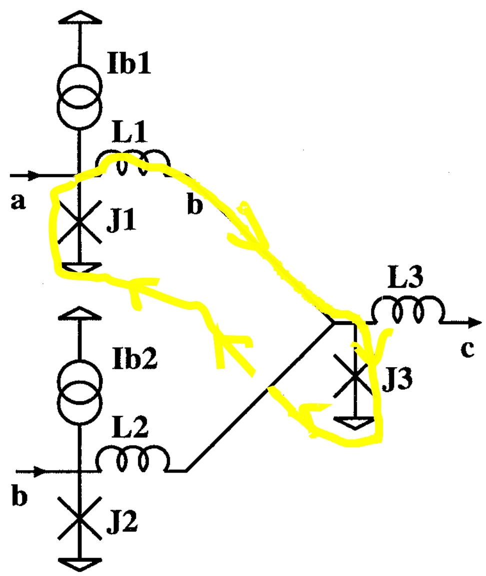


Figure 3. RSFQ implementation of 1×1-*Join*

5.2. New RSFQ Primitives

In this section we describe the RSFQ implementations of two more sophisticated DI primitives, the 1×2-*Join* and the *Tria*. We present results from circuit simulations and from testing of fabricated circuits as well as the principles of operation.

5.2.1. RSFQ 1×2-*Join*

An RSFQ implementation of the 1×2-*Join* is depicted in Fig. 6. The idea is to couple two 1×1-*Join* circuits shown above with a common row input a. Thus there are two interferometers $(J2, L2, J4)$ for column input $b0$ and $(J1, L1, J6, J4)$ for row input a coupled through $J4$ that act exactly as the 1×1-*Join* above with inputs a and $b0$ and output $c0$. There is a similar pair of interferometers, $(J3, L3, J5)$ for column input $b1$ and $(J1, L1, J7, J5)$ for row input a coupled through $J5$ with output $c1$. The two additional buffer junctions $J6$ and $J7$ isolate upper and lower parts of the circuit in a manner similar to that of junctions $J3$ and $J4$ of the *Merge* element.

The operation of the 1×2-*Join* is exemplified by the results of a computer simulation using the PSCAN program [15] depicted in Fig. 4. First, at time $t = 50$ picoseconds (pS), an input SFQ pulse arrives on $b0$ and trips b0 $J2$ (trace $V(J2)$), setting up a persistent current in the loop $(J2, L2, J4)$ (trace $I(L2)$). Then another input SFQ pulse arrives at $t = 100\ pS$ on a and switches junction a $J1$ (trace $V(J1)$), setting up persistent currents in loops $(J1, L1, J6, J4)$ and $(J1, L1, J7, J5)$ (trace $I(L1)$). The additive currents in the two loops that include $J4$ cause it to trigger (trace $V(J4)$) and produce an output SFQ pulse on $c0$. The 2π leap of $J4$ (trace $P(J4)$) steers the current into the low-inductance loop $(J4, J6, J7, J5)$. Proper selection of circuit parameters guarantees that this current switches $J7$ (trace $V(J7)$) so that the output $c1$ is isolated from the pulse generated at $c0$. Additionally, the switching of $J7$ resets the current in $(J1, L1, J7, J5)$, thus ensuring that the entire circuit is restored to its initial state. The rest of the simulation run shows an analogous process for the lower part of the circuit, but this time the row input a ($t = 150\ pS$) precedes column input $b1$ ($t = 200\ pS$).

It is important to note that the operation of the 1×2-*Join* is independent of the timing between the two input pulses. The circuit operates correctly even if these pulses arrive simultaneously. This is an important advantage compared with previously proposed RSFQ logic families [7, 13, 1, 6, 8, 10] since the latter all contain primitives which require some minimum delay between certain input events for correct operation. This difference in timing requirements is due to the fact that clocked RSFQ gates use a current controlled *pair* of Josephson junctions (a Josephson comparator) to implement various logical functions, while the asynchronous gates introduced in this work use superposition of persistent currents in a *single* junction for the same purpose.

A JJ is (unfortunately) a two terminal device with slightly greater than unit gain. As a result, logical elements

In the $J2-L2-J4$ loop, the loop current in $J2$ is substantially different than $J4$!

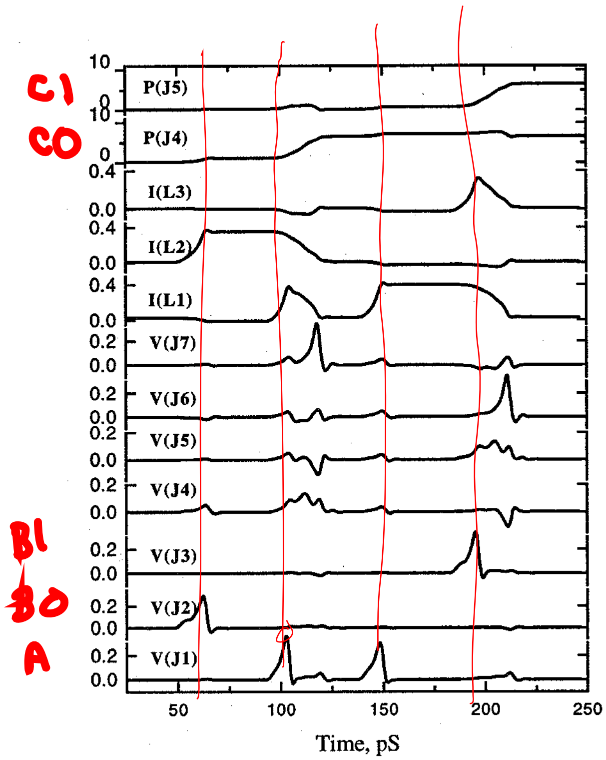


Figure 4. Computer simulation of 1×2-*Join*. All currents I() are in mA, all voltages V() - in mV. See Fig. 6 for the circuit parameter values used.

based on JJs suffer from poor operating margins. The poorest margins are usually associated with the critical currents of the JJs in decision-making current-controlled comparators (the other important parameters are the critical margins of dc bias current and the inductances). Thus designs like ours, which do not employ current-controlled comparators, can be expected to have relatively good parameter margins.

Among the variety of margin optimization techniques (e.g. N-dimensional design centering, Monte-Carlo yield optimization, etc.), we use the simplest one — maximization of minimum (critical) 1D margins. A 1D margin is the variance of one circuit parameter (say critical current) with all other circuit parameters held fixed. This technique is primitive but very fast. To account for correlated changes of circuit parameters we use the following three parameters: (1) XI — the simultaneous deviation of the dc biases, (2) XJ — the variation of the critical current density of the junctions, and (3) XL — the simultaneous deviation of the circuit inductances. Our approach to circuit optimization is described in detail in [14].

Optimization of the circuit parameters of the 1×2-*Join* shows that this is a very robust circuit. In particular, the margins on Josephson junction critical currents exceed $\pm 45 - 50\%$, and the bias current margin XI is well above $\pm 50\%$. It is interesting to note that for a "standard" Josephson transmission line (JTL) with junctions biased at 0.7 of their nominal critical current [13], the XI is $\pm 43\%$, so that we had to reoptimize the parameters of interconnect JTLs to match the XI of the 1×2-*Join* in order to measure the margins of this circuit.

We have experimentally confirmed the low-frequency operation of the 1×2-*Join* using the test circuit shown in Fig. 5 and implemented it in Hypres' 3.5 μ, 1 kA/cm^2 Nb-trilayer technology [4]. The experimental data are shown in Fig. 7. The input currents $I(a)$, $I(b0)$, $I(b1)$ are supplied by the room-temperature experimental setup described in [17]. These enter DC/SFQ converters that transform their rising edges into SFQ pulses. The latter propagate along the JTLs to the corresponding inputs of the 1×2-*Join* circuit shown in Fig. 6. In addition, RSFQ splitters (forks) are used to drive T flip-flop-based SFQ/DC converters [13] that monitor the correctness of circuit inputs. These converters toggle the output voltage each time they have an SFQ pulse at their inputs (traces $V(a)$, $V(b0)$, $V(b1)$ on Fig. 7). The same type of SFQ/DC converter monitors output pulses produced by the 1×2-*Join* (traces $V(c0)$ and $V(c1)$).

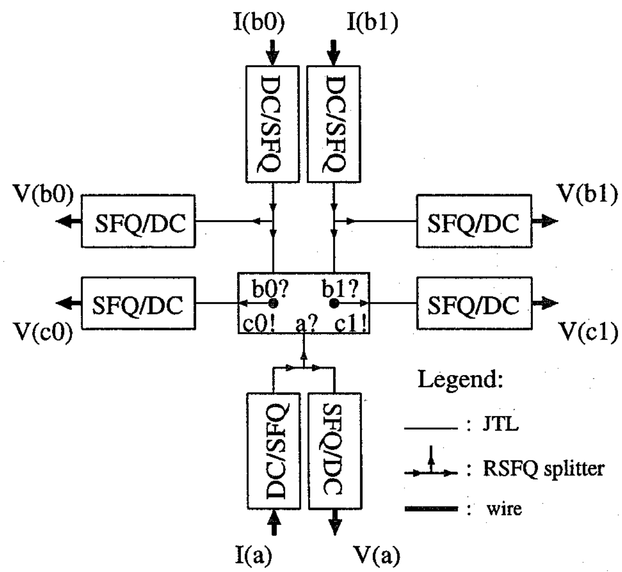


Figure 5. Block-diagram of 1×2-*Join* test circuit.

5.2.2. RSFQ *Tria* and 2×2-*Join*

An RSFQ implementation of the *Tria* is shown in Fig. 8. It is closely related to the implementations of the 1×1-*Join* and 1×2-*Join* described above. It can be thought of as a

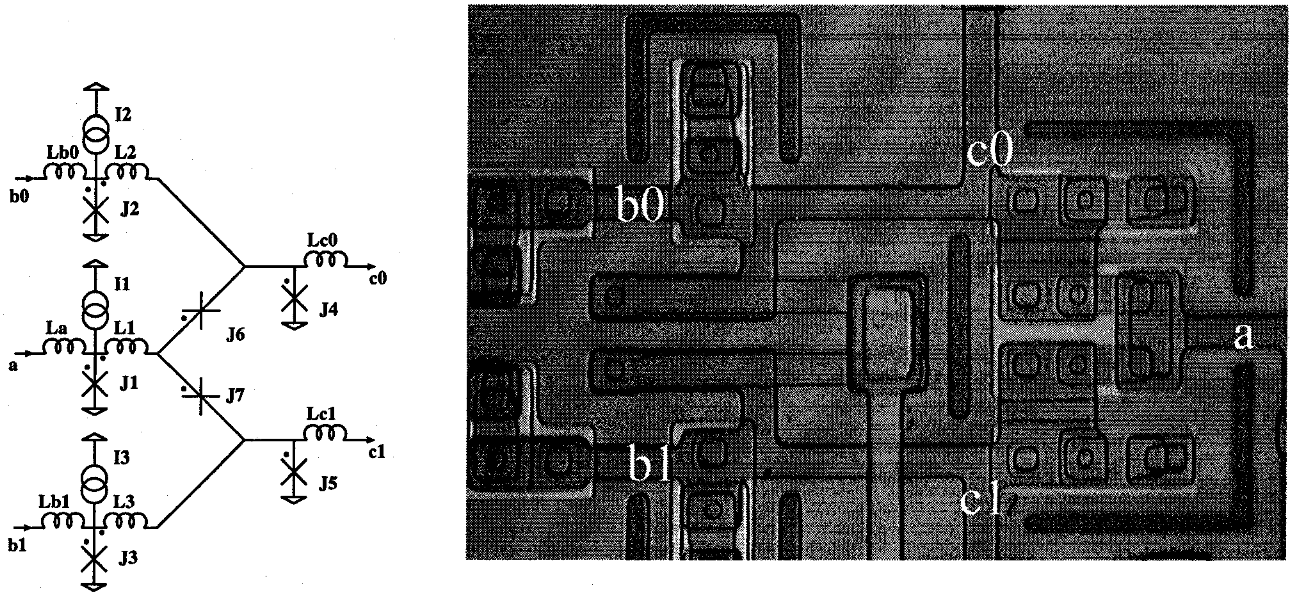


Figure 6. RSFQ implementation of 1×2-*Join*. Dots on circuit element terminals specify the polarity of currents and voltages at these elements: positive current (voltage) flows from (is applied between) a dotted terminal to (and) the undotted one. Optimized parameters: $I1 = 0.09\ mA, I2 = I3 = 0.07\ mA; La = 2.2\ pH, Lb0 = Lb1 = 2.0\ pH, L1 = 2.4\ pH, L2 = L3 = 3.5\ pH, Lc0 = Lc1 = 3.6\ pH; J1 = 0.19\ mA, J2 = J3 = 0.29\ mA, J4 = J5 = 0.24\ mA, J6 = J7 = 0.26\ mA$; **for all junctions** $\beta_c = 1$, $V_c = 0.3\ mV$. **The input/output terminals are connected to JTLs with the following parameters: critical current of the Josephson junctions** $Jc = 0.25\ mA$, **inductance between the junctions** $L = 3\ pH$, **bias current of the junctions** $Ib = 0.3\ mA$. **A microphotograph of a fabricated** 1×2-*Join* **cell is shown on the right.**

circular closure of three 1×1-*Joins* with additional buffer junctions protecting the *Tria* outputs in a manner similar to that of 1×2-*Join*.

Suppose the *Tria* first receives an input SFQ pulse on a which trips $J1$ and induces persistent currents in a pair of interferometers $(J1, L1, J4, J7)$ and $(J1, L1, J5, J10)$. If an input pulse arrives at b, then $J2$ switches and induces persistent currents in another pair of interferometers $(J2, L2, J6, J8)$ and $(J2, L2, J9, J7)$. Among the Josephson junctions of four interferometers with induced persistent currents there is only one, $J7$, where these currents of two interferometers $(J1, L1, J4, J7)$ and $(J2, L2, J9, J7)$ are added. The choice of circuit parameters guarantees that this sum exceeds the critical current of $J7$. As a result, $J7$ trips and produces an output pulse on p, also resetting the current in all loops containing $J7$. The switching of $J7$ also results in the persistent current that was earlier split between interferometers $(J1, L1, J4, J7)$ and $(J1, L1, J5, J10)$ being diverted exclusively into $(J1, L1, J5, J10)$. Again, the choice of circuit parameters guarantees that this current exceeds the critical current of $J5$. The switching of $J5$ protects output q and input c and also resets current in the interferometer $(J1, L1, J5, J10)$. A similar process results in the switching of $J6$, resetting the current in $(J2, L2, J6, J8)$. With no persistent currents in the *Tria*'s interferometers, the entire circuit returns to the initial state.

The approach used in *Tria* can also be extended to larger *Joins*, such as the 2×2-*Join* depicted in Fig. 9. This circuit was simulated and optimized as shown in Fig. 10. At the optimum point, the margins on the Josephson junctions exceeded $\pm 34\%$ and the margin on the bias current was $\pm 30\%$. The circuit's low-frequency operation was experimentally confirmed using a test approach analogous to that for the 1×2-*Join* (see Fig. 11). The experimental data for the input sequence $b?c? - b?d? - a?c? - a?d?$ is presented in Fig. 12.

It is important to note that the schematics of *Joins* allow extremely regular and compact layout. For example, the central part of the 2×2-*Join* $(Jq, Jqc, Jqb, Js, Jsb, Jsd, Jr, Jra, Jrb, Jp, Jpc, Jpa)$

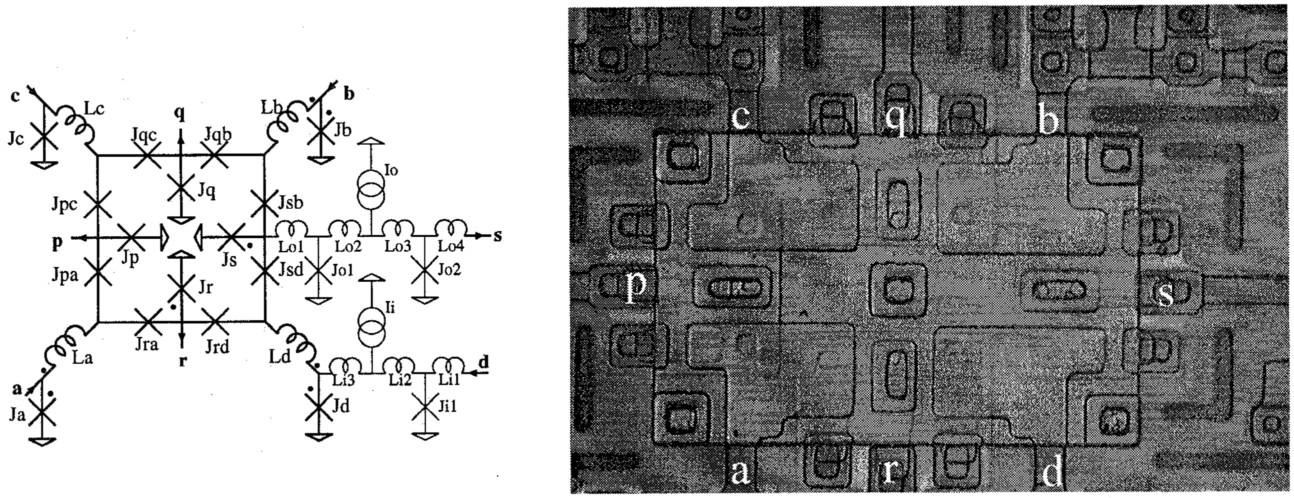


Figure 9. RSFQ implementation of 2×2-*Join*. The parameters of connecting JTL (thin line circuit elements shown only for input d and output r in order to make the schematics more compact) were changed to maximize circuit margins. Optimized parameters: $Ii = 0.15\ mA, Io = 0.26\ mA$; $La = Lb = Lc = Ld = 2.1\ pH, Li1 = 1\ pH, Li2 = Li3 = 0.9\ pH, Lo1 = 5.9\ pH, Lo2 = Lo3 = 1.3\ pH, Lo4 = 1.7\ pH$; $Ja = Jb = Jc = Jd = 0.21\ mA, Jp = Jq = Jr = Js = 0.14\ mA, Jqc = Jqb = Jra = Jrd = Jpa = Jpc = Jsb = Jsd = 0.25\ mA, Ji1 = 0.21\ mA, Jo1 = 0.12\ mA, Jo2 = 0.21\ mA$; **for all junctions** $\beta_c = 1$, $V_c = 0.3\ mV$. **Microphotograph of fabricated 2×2-*Join* cell is shown on the right.**

occupies area $75 \times 80\ \mu^2$ (see Fig. 9) when Hypres' $3.5\ \mu$ technology is used. Moreover, in CMOS implementations, a *Join* primitive with n outputs generally requires roughly n C-elements, n Xor or equivalent gates and roughly n inverters. For example, a single 2×2-*Join* needs about 4 each of the C-elements, $Xors$, and inverters — their connectivity making it even worse in layout.

5.3. Other Primitives

For universality, we need an arbitration circuit such as a *Mutex* or a *Sequencer* primitive. The design of arbiters in RSFQ is quite interesting and is currently under investigation.

RSFQ implementations of other relatively sophisticated stateholding devices, such as R-S and T flip-flops, are given in [7]. The T flip-flop given there can alternatively be considered to be an implementation of our *Toggle* primitive.

5.4. Comparison to RSFQ Boolean Gates

According to [7], implementation of a standard 2-input *Or* cell is characterized by 3 units of time delay, 9 JJs and 900 units of area. In contrast the numbers for a *Fork* cell are 1, 1, 210, and for a 1×1-*Join* cell they are 2, 3, 50. Our largest and most complex primitive, the 2×2-*Join*, has 3 units of time delay, 16 JJs and only 240 units of area. This compares very favorably with the Or, which is the Boolean gate most efficiently implementable in RSFQ.

6. A Design Example

As a small design example for illustration, consider the behavior of a full-adder described as $\mathbf{pref}((a\|b\|c)(sum\|carry))\star$. The specification's interpretation is that the adder's external behavior is an indefinite repitition of inputs a, b and c (carry-in) coming concurrently followed by the generation of concurrent outputs sum and $carry$. Another specification that might be suitable for a ripple carry adder is "eager" generation of the $carry$ in the sense that $carry$ is output as soon as its proper value can be determined from a & b. If we represent all signals in dual rail, then an implementation of the eager adder cell is described in trace theory as follows:

$$
\begin{aligned}
&\mathbf{pref}(((a0\|b0)\ \ carry0\ P) \mid ((a1\|b1)\ \ carry1\ P)\\
&\quad \mid (((a0\|b1) \mid (a1\|b0))\ \ Q))\star\\
&\quad \|\\
&\quad (((P\|c0)\ sum0) \mid ((P\|c1)\ sum1)\\
&\quad \mid ((Q\|c0)\ (sum1\|carry0))\\
&\quad \mid ((Q\|c1)\ (sum0\|carry1)))\star
\end{aligned}
$$

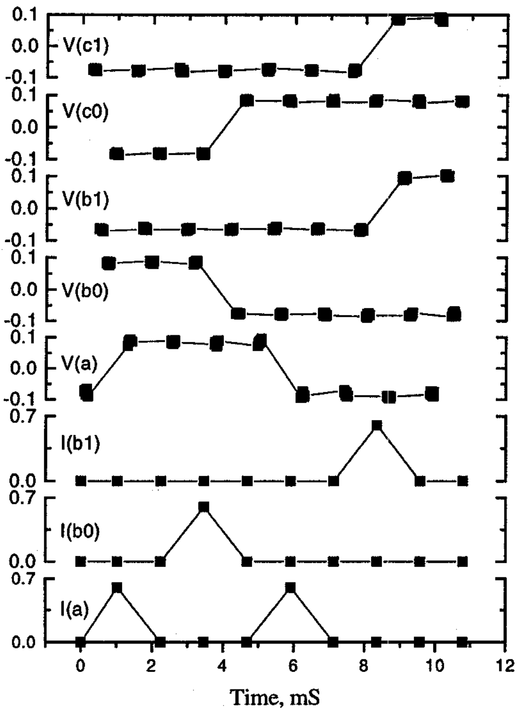


Figure 7. Low-frequency testing of 1×2-*Join* corresponding to the input sequence a?b0?-a?b1?. All currents I() are in mA, all voltages V() - in mV. Squares denote experimentally measured data while connecting lines are used only to simplify analysis of the graph.

In the above, *internal* symbols P and Q serve to synchronize various smaller concurrent processes within the specification.

The logic diagram of Fig. 13 shows how this specification may be implemented delay insensitively in RSFQ. A pulse on input wire $a0$ represents logic value 0 for input a of the adder, etc.

The figure shows two 2×2-*Join* devices: the one to the left receives the a and b inputs and generates the P and Q signals, and the right hand one combines the P and Q signals with the c inputs. The circuit employs 6 *Merge* gates and 4 *Fork* gates in addition to the 2×2-*Join* devices. Note that the bias currents and some of the inductances have been omitted from the figure for clarity. Due to the compactness the 2×2-*Join* implementation, the overall adder layout is expected to be quite small.

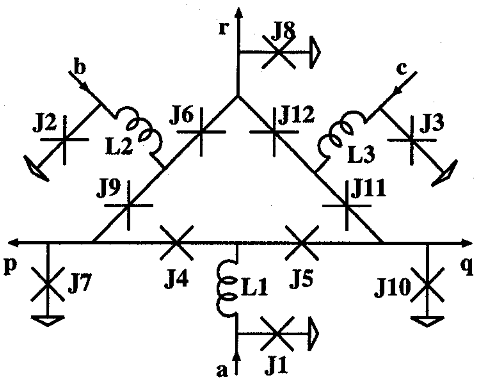


Figure 8. RSFQ schematic of *Tria*

7. Conclusion

We have demonstrated a novel, efficient design technique and some robust, compact and working implementations in RSFQ for crucial DI primitives that suffice to build sophisticated asynchronous circuits. These new circuits are at least comparable in area, speed, and parameter margins to existing implementations of Boolean primitives in RSFQ, in sharp contrast to the situation in CMOS. As building block of DI circuits, they have the critical advantage over existing clocked RSFQ logic elements that they require no minimum separation in the arrival times of input event to work correctly. We are currently working to complete the full RSFQ implementation of a universal set of DI primitives by implementing a synchronization circuit such as *Mutex* or *Sequencer*. We are also investigating design styles for RSFQ logic that blend DI techniques with other synchronous and asynchronous approaches.

It is interesting to discover that in RSFQ the event logic primitives of DI circuits are at no disadvantage relative to Boolean primitives as they are in CMOS and related technologies. In some respects, DI primitives appear to be more natural to implement in RSFQ than Boolean gates. The interesting possibility exists that ultra high speed computers of the future will be based on RSFQ superconducting technology [3], and that digital electronics implemented in RSFQ will make extensive use of DI primitives and related asynchronous circuits instead of relying exclusively on the ubiquitous synchronous Boolean logic of today's machines.

Acknowledgement

We thank Profs. K. K. Likharev, V. K. Semenov, and A. V. Rylyakov for stimulating discussions.

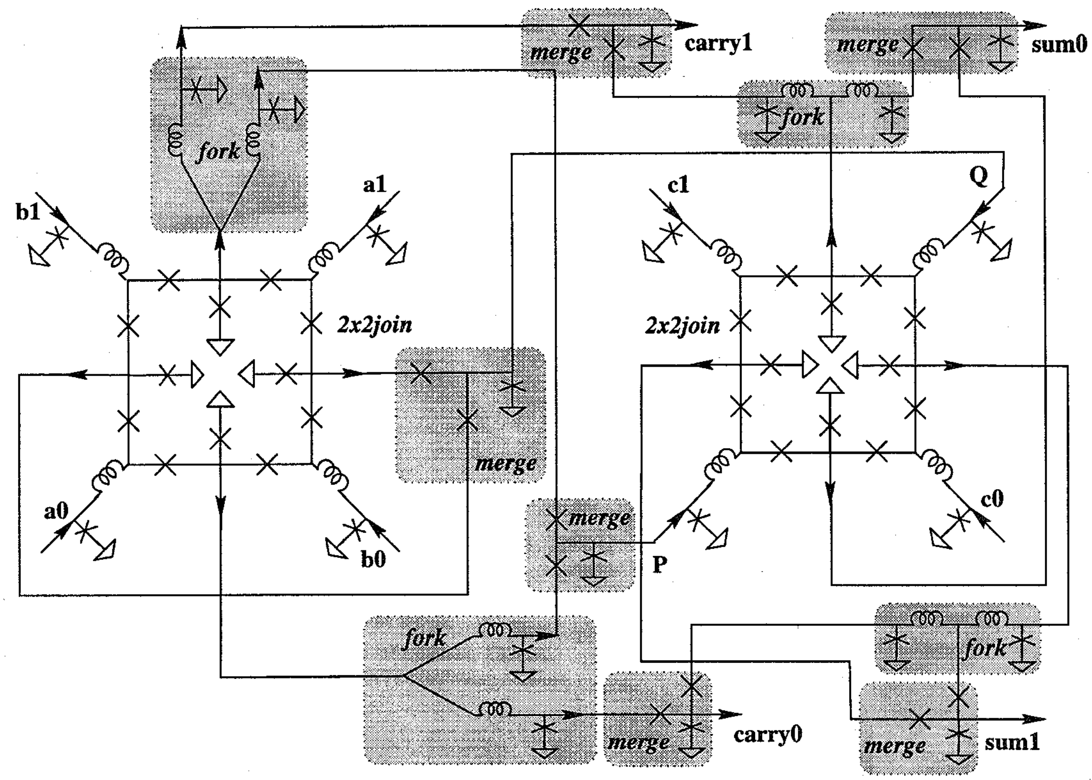


Figure 13. DI dual-rail implementation of full adder

References

[1] J. Z. Deng, S. R. Whiteley and T. Van Duzer. Data-Driven Self-Timing of RSFQ Digital Integrated Circuits. *Fifth International Superconductive Electronics Conference*, September, 1995, pp. 189–191.

[2] J. C. Ebergen. A formal approach to designing delay-insensitive circuits. *Distributed Computing*, 5(3):107–119, 1991.

[3] G. Gao, K. K. Likharev, P. C. Messina, T. L. Sterling. Hybrid Technology Multithreaded Architecture. *6th symposium on the Frontiers of Massively Parallel Computing*, October 27-31, 1996, Annapolis, MD.

[4] Hypres design rules. 175 Clearbrook Rd., Elmsford, NY 10523, phone (914)592-1190.

[5] R. M. Keller. Towards a theory of universal speed-independent modules. *IEEE Transactions on Computers*, C-23(1):21–33, Jan. 1974.

[6] I. Kurosawa, H. Nakagawa, M. Aoyagi, M. Maezawa, Y. Kameda, and T. Nanya. A basic circuit for asynchronous superconductive logic using RSFQ gates. *Supercond. Sci. Technol.*, 9(4A):46–49, Sept. 1995.

[7] K. Likharev and V. K. Semenov. RSFQ logic/memory family: a new Josephson junction technology for sub-teraherz clock frequency digital systems. *IEEE Trans. on Applied Superconductivity*, 1(1):3–28, Mar. 1991.

[8] M. Maezawa, I. Kurosawa, Y. Kameda, and T. Nanya. Pulse-driven dual-rail logic gate family based on rapid single-flux-quantum (RSFQ) devices for asynchronous circuits. *Second International Symposium on Advanced Research in Asynchronous Circuits and Systems*, 1996.

[9] A. J. Martin. The limitations to delay-insensitivity in asynchronous circuits. In W. J. Dally, editor, *Sixth MIT Conference on Advanced Research in VLSI*, pages 263–278. MIT Press, 1990.

[10] T. Nanya and Y. Kameda. Pulse-driven delay-insensitive circuits using single-flux-quantum devices. *Proc. of Int'l Conf. on Comp. Design*, Oct. 1996.

[11] P. Patra. *Approaches to Design of Circuits for Low-Power Computation.* PhD thesis, The University of Texas at Austin, 1995.

[12] P. Patra and D. S. Fussell. Efficient delay-insensitive rsfq circuits. *Proc. of Int'l Conf. on Comp. Design*, Oct. 1996.

[13] S. Polonsky, V. Semenov, P. Bunyk, A. Kirichenko, A. Kidiyarova-Shevchenko, O. Mukhanov, P. Shevchenko, D. Schneider, K. Likharev, and D. Zinoviev. New RSFQ circuits. *IEEE Trans. on Applied Superconductivity*, 3(1):2566–2577, Mar. 1993.

[14] S. Polonsky, V. Semenov, and A. Kirichenko, Single Flux Quantum B-Flip-Flop and its Possible Applications. *IEEE Trans. on Applied Superconductivity*, 4(1):9–18, Mar. 1994.

[15] S. Polonsky, V. Semenov, and P. Shevchenko. PSCAN: personal superconductor circuit analyzer. *Supercond. Sci. Technol.*, 4:667–670, 1991.

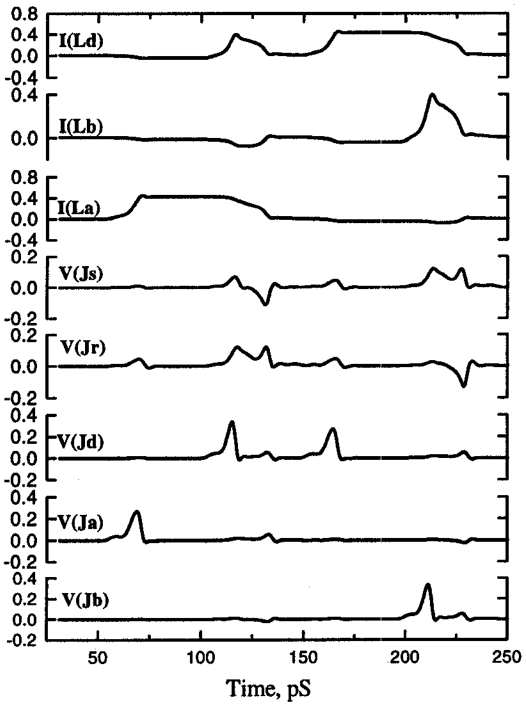


Figure 10. Computer simulation of 2×2-*Join*. Input sequence: $a?d? - d?b?$. All currents I() are in mA, all voltages V() - in mV.

[16] J. T. Udding. *Classification and Composition of Delay-Insensitive Circuits*. PhD thesis, Dept. of Math. and C.S., Eindhoven Univ. of Technology, 1984.

[17] D. Zinoviev and Y. Polyakov. Octopus: an advanced automated setup for testing superconductor circuits. *IEEE Trans. on Applied Superconductivity* (to appear).

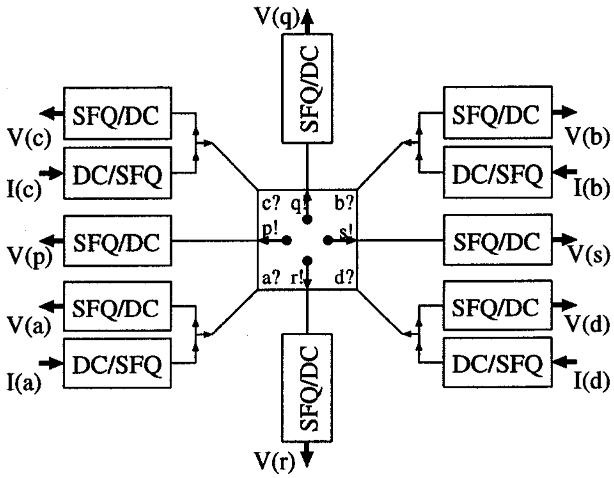


Figure 11. Block-diagram of 2×2-*Join* test circuit

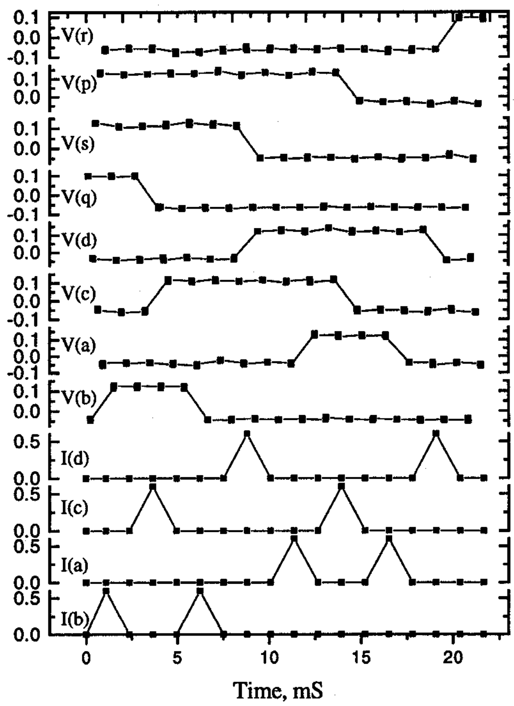


Figure 12. Low-frequency testing of 2×2-*Join* corresponding to the input sequence $b?c? - b?d? - a?c? - a?d?$. All currents I() are in mA, all voltages V() - in mV.